Clinical Trial Matching Pipeline Documentation

Pipeline Overview

This pipeline automates the matching of patients to clinical trials using a combination of AI-powered summarisation, hybrid vector search, and LLM reasoning. It operates in four phases and supports two retrieval directions — finding trials for a patient, and finding patients for a trial.

Four Phases at a Glance

- **Phase 1** Phase 1 — Data Preparation:
  - Fetch raw patient records from PostgreSQL and use Claude to structure them into clean 7-category profiles, saved as JSON.
- **Phase 2** Phase 2 — Ingestion:
  - Convert patient profiles and clinical trial records into dense + sparse (BM25) vectors and upsert into Qdrant vector database.
- **Phase 3** Phase 3 — Retrieval:
  - Run a hybrid RRF search in two directions: Patient to Trials and Trial to Patients.
- **Phase 4** Phase 4 — LLM Matching:
  - For each patient-trial pair above a score threshold, use an LLM to reason over inclusion and exclusion criteria.

Key Design Decisions

- Resumability — every script saves progress after each record, so restarts skip already-processed items.
- Hybrid search — dense embeddings (semantic) + BM25 sparse vectors (keyword) fused with Reciprocal Rank Fusion (RRF) for better recall.
- Two retrieval directions — patient-centric and trial-centric outputs serve different team use cases.
- Score thresholding — only pairs above a configurable score are sent to the LLM, controlling cost and speed.

Script 1 — block_summaries.py

Phase 1 Data Preparation

Purpose

Connects to the PostgreSQL database, retrieves raw patient records, and sends each one to Claude. Claude returns a structured 7-category JSON summary for every patient. Results are written incrementally so the script can be safely interrupted and resumed.

Inputs

- PostgreSQL table: mimic_patient_summary (patient_id, summary_text)
- CLI args: --max (record limit, default 1502), --output (output file path)
- Environment variable: ANTHROPIC_API_KEY

Key Steps

- Initialise Anthropic client and PostgreSQL connection
- Query the database with a configurable LIMIT
- Load any existing output JSON for resume support
- For each patient not already processed:
  - Send summary_text to Claude (claude-sonnet-4, temperature=0)
  - Parse and validate the JSON response against 7 required keys
  - Each value must be a plain string or null — no nested objects
  - On success: store the result and save to disk
  - On failure: store error, save, and exit cleanly

Output

- patient_block_summaries.json — dict of patient_id to structured profile
- 7 keys per patient: demographics, allergies, conditions, procedures, medications, observations, immunizations

Error Handling

- Strips markdown fences from Claude's response before parsing
- Falls back to regex extraction if direct JSON parse fails
- Validates key set and value types strictly — exits on first unrecoverable error

Script 2 — ingest_patients_block.py

Phase 2 Ingestion

Purpose

Reads the structured patient profiles from block_summaries.py output, converts each profile into dense and sparse vector representations, and upserts them into the Qdrant PATIENT_COLLECTION. Supports an optional expanded term cache to improve sparse retrieval quality.

Inputs

- patient_block_summaries.json — output from Script 1
- Optional: expanded_cache.json — pre-computed expanded query terms per patient
- CLI args: --json-path, --batch-size (default 50), --recreate, --expanded-cache

Key Steps

- Load patient profiles and optional expanded term cache
- Ensure Qdrant PATIENT_COLLECTION exists (recreate if --recreate flag is set)
- For each patient:
  - profile_to_dense_text() — convert profile into a single semantic string
  - expand_query_terms() — generate expanded medical terms (or load from cache)
  - Build PointStruct with stable UUID id, dense + sparse vectors, and payload
  - Payload includes: patient_id, raw_profile, _expanded terms
- Upsert in batches; sleep 0.1s between batches to avoid rate limits

Vector Strategy

- Dense vector: full semantic embedding of the profile text
- Sparse vector: BM25 over expanded medical terms for keyword recall
- Stable UUID from patient_id ensures idempotent upserts

Output

- Qdrant PATIENT_COLLECTION — populated with one point per patient

Script 3 — ingest_trials.py

Phase 2 Ingestion

Purpose

Reads clinical trial records from a JSONL corpus, builds dense and sparse vector representations for each trial, and upserts them into the Qdrant TRIAL_COLLECTION. This is the primary ingestion path for trials. A fallback version of this logic also exists inside retrieve_patients_block.py.

Inputs

- corpus.jsonl — one trial JSON object per line
- CLI args: batch_size (default 50), recreate flag

Key Steps

- Ensure Qdrant TRIAL_COLLECTION exists
- Stream corpus.jsonl line by line
- For each trial:
  - build_trial_text() — structured labeled string: title, diseases, summary, interventions, criteria
  - build_trial_keywords() — lean BM25 token string: title, diseases, drugs, criteria
  - Extract metadata: nct_id, title, min_age, max_age, sex from inclusion criteria
  - Build PointStruct with hash(nct_id) as id
- Upsert in batches of 50; sleep 0.2s between batches

Payload Fields

- nct_id, title — trial identifiers
- min_age, max_age, sex — extracted from inclusion criteria text for filtering
- dense_text, bm25_text — stored for inspection and re-use
- inclusion_criteria, exclusion_criteria — raw text for LLM reasoning

Output

- Qdrant TRIAL_COLLECTION — populated with one point per trial

Script 4 — retrieve_patients_block.py

Phase 3 Retrieval — Direction A: Patient to Trials

Purpose

For each patient, queries the Qdrant TRIAL_COLLECTION using three parallel prefetch queries fused with Reciprocal Rank Fusion (RRF). Returns a ranked list of the most relevant trials per patient. Also contains a fallback ingestion path for trials in case ingest_trials.py was not run separately.

Inputs

- corpus.jsonl — clinical trials (used for optional fallback ingestion)
- patient_block_summaries.json — patient profiles
- CLI args: --corpus, --patients, --output, --top-k (default 10), --threshold (default 0.1), --recreate, --no-ingest

Key Steps

- If --no-ingest is not set: ingest trials into TRIAL_COLLECTION (fallback path)
- Load patient profiles from JSON
- For each patient:
  - Fetch _expanded terms from PATIENT_COLLECTION by stable UUID lookup
  - Build three query vectors: dense(base_text), BM25(base_text), BM25(expanded)
  - Prefetch 150 candidates each from TRIAL_COLLECTION
  - Fuse results with RRF and return top_k
  - Filter by score >= threshold
- Save all results to output JSON

Retrieval Strategy — RRF Fusion

- Dense prefetch — semantic similarity between patient profile and trial text
- Sparse base prefetch — keyword overlap on raw profile text
- Sparse expanded prefetch — keyword overlap on medically expanded terms
- RRF fusion combines all three rankings for robust recall

Output

- retrieved.json — dict of patient_id to raw_profile + list of scored trials
- Each trial entry: nct_id, title, RRF score

Script 5 — matching_blocks.py

Phase 4 LLM Matching — Direction A

Purpose

Takes the retrieved patient-trial pairs from retrieve_patients_block.py and runs LLM-based reasoning over each pair. For every trial above the score threshold, the LLM evaluates each inclusion and exclusion criterion individually and returns a structured eligibility result.

Inputs

- retrieved.json — output from Script 4
- corpus.jsonl — used to look up full trial details by nct_id
- CLI args: --retrieved, --corpus, --output, --model, --threshold (default 0.1), --data_mode (full or filtered)

Key Steps

- Load retrieved results and corpus into memory
- Load existing output for resume support
- Select patient IDs based on data_mode:
  - filtered — only process TARGET_PATIENTS (predefined subset)
  - full — process all patients in the retrieved results
- For each patient:
  - Build patient_text from raw_profile fields + CONSENT_SENTENCE appended
  - Filter trials by score >= threshold
  - For each trial: look up full trial info from corpus, call reason_trial()
  - reason_trial() sends patient + trial to LLM and returns criteria-level reasoning
- Save results to output JSON after each patient

Output

- reasoning_results.json — patient-centric match results
- Structure: results[patient_id][0][nct_id] = reasoning result
- Each result contains per-criterion inclusion/exclusion evaluations

Script 6 — trial_2_patient_blocks.py

Phase 3+4 Retrieval + LLM Matching — Direction B: Trial to Patients

Purpose

The reverse direction of the pipeline. For each trial (or a single specified trial), queries the Qdrant PATIENT_COLLECTION to retrieve the most eligible patients. Immediately follows retrieval with inline LLM reasoning over inclusion and exclusion criteria for every matched patient. Combines Phase 3 and Phase 4 in a single script.

Inputs

- corpus.jsonl — source of trial records
- Qdrant PATIENT_COLLECTION — must already be populated by Script 2
- CLI args: --corpus, --nct-id (single trial) or --all, --top-k (default 20), --threshold (default 0.1), --retrieved, --reasoning

Key Steps

- Load trials: single by --nct-id or all from corpus using --all
- Load existing retrieved and reasoning JSONs for resume support
- For each trial not already processed:
  - build_trial_entry() — extract title, phase, drugs, diseases, criteria, NCTID
  - retrieve_patients() — query PATIENT_COLLECTION with dense + sparse prefetch, fuse with RRF
  - Filter patient hits by score >= threshold
  - Store retrieval results in t2p_retrieved.json
  - For each matched patient: call reason_trial() inline
  - Store reasoning in t2p_reasoning.json
- Save both output files to disk after every trial

Key Difference from Direction A

- Retrieval and reasoning happen in a single script (not two separate ones)
- Query direction is reversed: trial vectors query the patient collection
- Designed for the use case: given a trial, who are the best candidate patients?

Output

- t2p_retrieved.json — trial-centric retrieval: nct_id to trial_entry + scored patient list
- t2p_reasoning.json — trial-centric reasoning: nct_id to per-patient criteria evaluation

Full Pipeline — End to End

Data Flow Summary

Step 1: block_summaries.py — PostgreSQL records structured by Claude into patient_block_summaries.json

Step 2a: ingest_patients_block.py — patient profiles vectorised and loaded into PATIENT_COLLECTION

Step 2b: ingest_trials.py — clinical trials vectorised and loaded into TRIAL_COLLECTION

Step 3A: retrieve_patients_block.py — hybrid RRF search per patient → retrieved.json

Step 4A: matching_blocks.py — LLM criteria reasoning → reasoning_results.json

Step 3B+4B: trial_2_patient_blocks.py — hybrid search + inline LLM reasoning → t2p_retrieved.json + t2p_reasoning.json

Execution Order

- Run block_summaries.py first — all downstream scripts depend on its output
- Run ingest_patients_block.py and ingest_trials.py before any retrieval script
- Direction A: run retrieve_patients_block.py, then matching_blocks.py
- Direction B: run trial_2_patient_blocks.py (retrieval and reasoning in one step)
- Directions A and B are independent — both can be run, or just one as needed

Shared Utilities

- embed() — generates dense vector embeddings
- bm25_sparse_vector() — generates sparse BM25 vectors
- expand_query_terms() — expands medical terms for improved sparse recall
- ensure_collection() — creates or recreates Qdrant collections
- extract_min_age(), extract_max_age(), extract_sex() — parse criteria metadata
- reason_trial() — LLM call for per-criterion eligibility reasoning (from matching_utils)

Resumability

All scripts support safe interruption and restart. Each saves progress to disk after every record (patient or trial). On restart, already-processed records are detected and skipped. No data is lost if a script is stopped mid-run.

Final Outputs

- reasoning_results.json — patient-centric: best matching trials per patient with LLM reasoning
- t2p_reasoning.json — trial-centric: best matching patients per trial with LLM reasoning